

Ranking Clarity Smart Contracts for a Package Registry on Stacks: Post-Conditions in Place of ERC-20 Allowances

Taehong Kwon

University of South Africa

Manuscript draft, 7 October 2026

Abstract

Every Clarity smart contract deployed on Stacks is stored on chain as readable source, yet developers have no registry comparable to npm or pkg.go.dev for finding, judging and reusing these contracts. Such a registry needs a ranking that keeps clones, test deployments and spam below the contracts people actually depend on, and that an attacker cannot buy cheaply with fake identities. Earlier work ranked Ethereum wallets with PageRank over ERC-20 transfers (AWP) or over ERC-20 allowances (EndorseRank). Stacks has no allowance. The SIP-010 token standard has no `approve` or `transferFrom`; a user lets a contract move their tokens by calling it and bounds what may leave their account with post-conditions on the signed transaction, and since Clarity 4 a contract bounds its own outflows with asset allowances in `as-contract?` and `restrict-assets?`.

This paper maps the allowance relation onto these mechanisms and defines PC-EndorseRank, a ranking whose only nodes are contracts. Post-condition authorizations, reduced to their latest state per wallet, contract and asset as in EndorseRank, decide how each wallet divides its endorsement among contracts. The fees a wallet has paid decide how much that endorsement counts. Delegation and dependency references in contract code carry the score from contract to contract. We show that PageRank with a uniform restart on a wallet–contract graph reduces to a fractional count of wallets, so it offers no protection against Sybil wallets, and we bound the score that a group of attacker contracts can collect by its share of the restart and the score flowing into it from honest contracts. A synthetic example illustrates both results. We then describe the registry and its command-line client, `cpm`, which separates referencing a deployed contract from copying its source, and we set out an evaluation on Stacks mainnet, fixed before its label window opens, that measures attacker cost, prospective adoption and clone suppression against count-based, dependency-PageRank, AWP and EndorseRank baselines.

Keywords: Stacks; Clarity; smart contracts; package registry; PageRank; Sybil resistance; post-conditions; software supply chain

1 Introduction

Clarity, the smart contract language of Stacks, is interpreted rather than compiled, and a deployed contract is stored on chain as the source text that the node executes [1, 2]. Anyone can therefore read any contract on Stacks mainnet through a block explorer or the Stacks Blockchain API [3], and a public repository mirrors the source of every deployed contract on GitHub [4]. On Ethereum, by contrast, the source of a contract is visible only when its developer has chosen to publish and verify it. Developers who want to build on an existing Clarity contract have two options. They can call the deployed contract by its principal with `contract-call?`, or they can copy its source and deploy a new instance under their own address. Clarinet, the standard development tool, already supports the first option: `clarinet requirements add` downloads a mainnet contract into a project so that local tests can call it [5].

What Stacks lacks is the layer that npm, crates.io and pkg.go.dev provide on top of their code: search, documentation, version lineages, publisher identity, and an ordering that places the contracts other people rely on above copies, tests and spam. Package registries usually order search results with a popularity signal. The Go package site, for example, multiplies the text relevance of a module by the logarithm of the number of packages that import it [6]. Such counts are cheap to inflate. In April 2024 Sonatype reported about 14,000 npm packages published from several accounts to collect rewards from the tea protocol [7], which scores open-source packages by their position in the dependency graph [8]. By November 2025 Amazon Inspector had tied more than 150,000 npm packages to the same campaign; the packages declared dependencies on each other, so that installing one pulled in the rest [9]. A Stacks registry whose ranking decides which contract a developer installs will attract the same behavior, and the damage from a bad recommendation is larger, because a referenced contract can hold or move real assets.

Earlier work ranked Ethereum wallets with PageRank. Adaptive Weighted PageRank (AWP) weights ERC-20 transfers by their value and by a logistic decay in their age [10]. EndorseRank weights each owner–spender edge by the latest ERC-20 allowance between them, on the view that an approval is an explicit grant of spending authority rather than a payment that may have many motives [11]. This paper asks whether the same idea can rank Clarity contracts. Two problems stand in the way.

The first problem is that Stacks has no allowance. The SIP-010 fungible token trait defines transfer but no approve, allowance or transferFrom [12]. A user lets a contract move their tokens by calling that contract directly; the contract acts within the user's own transaction, and the user limits what may leave their account with post-conditions, which the protocol checks after execution [13]. Since Clarity 4 a contract can also limit its own outflows while it runs code it does not control, through the asset allowances of `as-contract?` and `restrict-assets?` [14]. These mechanisms carry the same information as an allowance, but they are scoped to a transaction or to a block of code rather than stored as a standing balance. Section 4 works out which of them can play the role of the allowance edge and how.

The second problem is that PageRank alone does not stop Sybil attacks. Cheng and Friedman proved that no reputation function that treats node identities symmetrically can be Sybil-proof [15], and link farms raise the PageRank of a target page by surrounding it with pages that link to it [16]. Resistance comes from asymmetry, as in TrustRank and SybilRank, where the walk starts or restarts only at trusted nodes [17, 18]. We show in Section 5 that on a graph in which wallets point to the contracts they authorize, PageRank with a uniform restart gives every wallet the same weight, so the score of a contract becomes a fractional count of the wallets that use it. Moving EndorseRank to Stacks without changing the restart would therefore be about as easy to game as counting callers.

The paper makes four contributions.

- **A mapping of the allowance relation to Stacks:** we compare ERC-20 approvals with post-conditions, Clarity 4 asset allowances, legacy `as-contract`, extension maps and static references, and we choose post-conditions for wallet-to-contract endorsements and code-level delegation and dependency for contract-to-contract edges. We state which post-conditions count as authorizations. AWP needs no replacement, because Stacks records asset transfers.
- **PC-EndorseRank:** a ranking over contracts only, in which wallets enter through the restart distribution. It keeps the latest-state rule of EndorseRank, weights each wallet by the fees it has paid, and propagates score along delegation and dependency edges. Three propositions describe its behavior under Sybil attacks, and a synthetic example illustrates them.

- **A registry and command-line client:** the Clarity Registry and `cpm` separate referencing a deployed contract (`cpm add`) from copying its source (`cpm create`, `cpm fork`), take publisher scopes from deployer addresses and BNS names, treat versions as lineages declared by the publisher, group clones by normalized source hashes and `contract-hash?`, and pin code hashes in a lockfile.
- **An evaluation protocol fixed in advance:** it compares PC-EndorseRank with raw counts, dependency PageRank, AWP and EndorseRank with a uniform restart on Stacks mainnet, measuring the cost an attacker pays to reach the top of the ranking, agreement with adoption in a later window, and the share of clones among search results. Raw counts are included because in our ERC-20 study they matched held-out approvals better than both PageRank variants [11].

Section 2 describes Stacks, Clarity and the mechanisms that control asset movement, together with the PageRank and registry background. Section 3 reviews related work. Section 4 maps allowances to Stacks, Section 5 defines PC-EndorseRank, and Section 6 describes the registry and `cpm`. Section 7 sets out the evaluation, Section 8 discusses limitations, and Section 9 concludes.

2 Background

2.1 Stacks and Clarity

Stacks is a blockchain for smart contracts that settles to Bitcoin [19]. Its contracts are written in Clarity, a language without recursion or unbounded loops, so the cost of a call can be bounded before it runs [1]. Clarity is not compiled: the node stores and interprets the source text, and the Stacks Blockchain API returns that text, together with the contract’s interface (ABI), for any deployed contract [3].

A contract is identified by a contract principal of the form `SP...ABC.amm-pool`. The address before the dot is the standard principal that deployed the contract, so the identifier itself records who published the code. A contract calls another with `contract-call?`. In *static dispatch*

the callee is written as a literal principal; in *dynamic dispatch* the callee is a function argument whose type is a trait, and the caller chooses the concrete contract at run time [1]. A trait is declared with `define-trait`; a contract states that it conforms to a trait with `impl-trait` and imports a trait for use as an argument type with `use-trait`. Both refer to the contract that defines the trait. Clarity 5 also accepts a constant as the target of `contract-call?` [20]. Static dispatch therefore yields a call graph that can be read from the source, while dynamic dispatch leaves the callee open until a transaction supplies it.

Token standards are traits. SIP-009 defines non-fungible tokens [21], SIP-010 fungible tokens [12] and SIP-013 semi-fungible tokens [22]. The Stacks Blockchain API can list the contracts whose ABI matches a given trait [3], which lets a registry label conforming contracts without parsing them.

2.2 How Stacks authorizes asset movement

No allowance in SIP-010. The SIP-010 trait declares a transfer function that takes an amount, a sender, a recipient and an optional memo, together with read-only functions for the name, symbol, decimals, balance, supply and token URI [12]. There is no function that lets one principal spend another's tokens. Typical implementations accept a transfer only when the sender is the transaction sender. In Clarity, `tx-sender` stays equal to the account that signed the transaction through nested contract calls, unless a contract switches the context to itself, while `contract-caller` is the immediate caller [1]. A decentralized exchange called by a user can therefore move the user's tokens within that call without any prior approval. What protects the user is the post-condition mechanism.

Post-conditions. A Stacks transaction carries a list of post-conditions and a post-condition mode, both covered by the signature [13]. Each post-condition names a principal (the transaction origin, a standard principal or a contract principal), an asset (STX, a fungible token or a specific non-fungible token) and a condition. For STX and fungible tokens the condition compares the amount the principal sends during the transaction with a stated amount using `=`, `>`, `≥`, `<` or `≤`; for a non-fungible token it states that the principal will or will not send it. After execution the node checks every post-condition; if one fails, the transaction is aborted and its effects are rolled back, although it stays in the block and its fee is paid. The mode decides what

happens to transfers that no post-condition mentions: in *Allow* mode they are permitted, in *Deny* mode they abort the transaction [13]. Stacks.js builds transactions in Deny mode unless told otherwise [23]. SIP-040, ratified with Clarity 5 for Epoch 3.4, adds a third mode, *Originator*, which applies Deny rules to the assets of the transaction origin and Allow rules to every other principal, and adds a *may send* condition for non-fungible tokens [24, 20]. Its motivation is that DeFi calls move assets between contracts in ways the user cannot predict, which pushed users towards Allow mode.

Asset allowances in contracts. Clarity 4 (SIP-033), activated at Bitcoin block 923,222 in November 2025, lets a contract bound asset outflows while it runs code it does not control [14]. `restrict-assets?` takes a principal, a list of allowances and a body; it executes the body and then checks the principal’s outflows against the allowances. `as-contract?` switches `tx-sender` and `contract-caller` to the contract itself, executes the body, and checks the contract’s own outflows. The allowance expressions are `with-stx`, `with-ft`, `with-nft` and `with-stacking`, each naming an asset and a limit; `with-all-assets-unsafe`, allowed only in `as-contract?`, lifts every limit. If an outflow exceeds an allowance, the body’s effects are rolled back and the expression returns the index of the violated allowance as an error; if an asset with no allowance moves, it returns `(err u128)`. Clarity 4 removes the old `as-contract`, which switched context without any limit. Listing 1 shows the pattern. SIP-033 also adds `contract-hash?`, which returns the SHA-512/256 hash of a contract’s code body so that one contract can check that another follows a known template, and the keyword `current-contract`.

Listing 1: A Clarity 4 asset allowance. Only the vault’s owner may trigger a payout, and while the body runs at most amount of the token may leave the vault.

```
(define-constant OWNER tx-sender)
(define-public (pay-out (amount uint) (recipient principal))
  (begin
    (asserts! (is-eq contract-caller OWNER) (err u401))
    (as-contract? ((with-ft .token "tok" amount))
      (try! (contract-call? .token transfer amount current-contract
        recipient none))))))
```

Standing authorizations between contracts. Some protocols keep explicit lists of contracts they trust. In the ExecutorDAO pattern, used by several Stacks DAOs, the core contract keeps a map of enabled extensions, a governance call to `set-extension` enables or disables one,

and an enabled extension may act in the DAO’s sending context, checked through `contract-caller` [25]. Such a map is the closest Stacks analogue of a standing allowance: it is set by an explicit transaction, it stays in force until changed, and it lets one contract use another’s authority.

2.3 Package registries and popularity

Package registries index published code and rank search results with some mix of text relevance and popularity. The Go package site computes, for each module matching a query,

$$\text{score} = \text{ts_rank}(q, m) \cdot (\ln(e + \text{imported_by}(m)))^\alpha \cdot \rho_{\text{redist}}(m) \cdot \rho_{\text{mod}}(m), \quad (1)$$

where `ts_rank` is PostgreSQL’s full-text rank, `imported_by` counts importing packages, and the last two factors penalize modules that cannot be redistributed or lack a `go.mod` file [6]. The logarithm makes a doubling of importers add a constant rather than double the score. Dependency networks of npm, Cargo, CRAN and other ecosystems have been studied as graphs [26, 27], and PageRank over the npm dependency graph has been used to find packages in decline [28].

The tea protocol paid rewards according to `teaRank`, a score of a package’s place in the dependency graph [8]. The incidents described in Section 1 show what happens when such a score carries money: spam packages that depend on one another [7, 9]. Registries face other supply-chain attacks as well, including typosquatting and account takeover [29, 30, 31], and a small number of maintainer accounts can reach a large share of npm through transitive dependencies [32].

2.4 PageRank, the restart distribution and Sybil attacks

Let $G = (V, E, w)$ be a directed graph with positive edge weights, and let P be the row-stochastic matrix with $P_{uv} = w(u, v) / \sum_x w(u, x)$ for nodes with out-edges. With damping $d \in (0, 1)$ and a restart distribution π over V , PageRank is the fixed point of

$$\mathbf{r} = (1 - d) \pi + d P^\top \mathbf{r} + d \delta(\mathbf{r}) \pi, \quad \delta(\mathbf{r}) = \sum_{v: \text{no out-edges}} r_v, \quad (2)$$

so that the mass held by dangling nodes returns to the restart distribution [33, 34]. The original algorithm uses a uniform π . Topic-sensitive and personalized PageRank use a non-uniform one [35, 36], and TrustRank concentrates it on a small set of pages judged by hand to be reputable [17]. EigenTrust uses pre-trusted peers in the same role [37]. AWP and EndorseRank use $d = 0.85$ and a uniform restart [10, 11].

A Sybil attack creates many identities controlled by one party [38]. Against PageRank it takes the form of a link farm, a set of pages that exist to link to a target; alliances of farms raise the scores of all participants [16]. Cheng and Friedman showed that a reputation function that is symmetric in node identities cannot be Sybil-proof, and that asymmetric, flow-based functions computed from a trusted source can be [15]. Social-graph defenses such as SybilGuard and SybilRank rest on the same observation: an attacker can create nodes and edges among them freely, but the number of *attack edges* from honest nodes into the attacker’s region is limited, and a walk that starts at honest nodes rarely crosses them [39, 18]. Analyses of PageRank on communities make the dependence on inbound links precise [40], and Robust PageRank limits how much of a node’s score can come from any small set of contributors [41]. Section 5 applies these ideas to contracts.

3 Related work

3.1 Graph-based ranking on blockchains

Do and Do computed PageRank and HodgeRank scores for Ethereum addresses from the transaction graph, with transferred value as edge weight [42]. AWP refines this with a logistic decay so that recent transfers count more than old ones [10]. RiskProp propagates risk labels through a bipartite transaction graph to rate accounts [43]. Chen et al. built three Ethereum graphs, for money flow, contract creation and contract invocation, and analyzed them with PageRank and other measures; their invocation graph is the closest earlier object to the contract graph we rank, although their aim was measurement rather than recommendation [44]. EndorseRank replaced transfers with the latest ERC-20 allowance between each owner and spender [11]. In that study most spenders turned out to be contracts, so the allowance graph already pointed mostly from

wallets to contracts. Address-clustering heuristics, some of them based on approvals, group Ethereum addresses that are likely to share an owner [45]; we return to such heuristics when discussing wallet weights.

All of these methods rank addresses for credit or risk. None of them asks which contracts a developer should reuse, and none studies what an attacker must pay to move a contract up the ranking.

3.2 Dependency networks and registry security

Studies of package ecosystems treat dependencies as a directed graph and measure its growth and fragility [26, 27]. Mujahid et al. tracked the PageRank of npm packages month by month and used falling centrality to flag packages in decline [28]. Registry security work catalogs attacks on open-source supply chains, among them typosquatting, malicious maintainers and account takeover [29, 30, 31], and shows how far a compromised maintainer account can reach through transitive dependencies [32]. Signing systems such as Sigstore bind a published artifact to an identity [46]. On Stacks the binding between a contract and its publisher comes for free, since the deployer’s address is part of the contract identifier, but a registry still has to decide how names, versions and scopes relate to that address.

3.3 Code clones in smart contracts

Copying is common in smart contracts. Kondo et al. found that 79.2% of the verified Ethereum contracts they studied were clones [47], and He et al. found duplicates for more than 96% of contracts in a set of about ten million [48]. Clones also serve fraud: copies of popular tokens are used to trick traders on decentralized exchanges [49]. Clone detection techniques range from exact hashes to token- and tree-based similarity [50]; near-duplicate detection at scale often uses MinHash over shingles [51]. A Stacks registry can add an exact on-chain check, because `contract-hash?` returns the hash of any contract’s code [14].

3.4 Tools for finding Clarity contracts

Several pieces of a registry already exist. The Stacks Blockchain API returns source code and ABIs and lists contracts by trait [3]. Clarinet downloads mainnet contracts as project requirements and remaps their addresses for local networks [5]. A public GitHub repository mirrors the source of deployed contracts [4], and Chainhooks streams chain events to applications [52]. STACY, a static analyzer for Clarity, flags patterns such as calls to other contracts inside `as-contract` [53]. None of these tools ranks contracts, groups clones, records versions or separates referencing a contract from copying it.

Taken together, earlier work offers ranking methods for addresses and measurement of dependency graphs, and Stacks offers the raw material for a registry. What is missing is a ranking of deployed contracts that uses an explicit authorization signal, that can be computed on a chain without allowances, and whose resistance to Sybil attacks is analyzed rather than assumed.

4 From ERC-20 allowances to Stacks

4.1 What EndorseRank takes from an allowance

EndorseRank builds a graph G_{approve} in which an edge $u \rightarrow v$ carries the latest allowance that owner u has granted to spender v , summed over tokens, and runs weighted PageRank on it [11]. Five properties of the ERC-20 allowance make this work:

- **An explicit act by the asset owner:** the owner signs an approve transaction or a permit message [54, 55].
- **A named counterparty:** the approval names the spender, usually a router or vault contract.
- **A bound per asset:** the allowance is an amount of one token.
- **A standing state:** the allowance stays in force until the owner changes it, and a new approval overwrites the old one. EndorseRank keeps only the latest value for each owner, spender and token, and needs no time decay.

- **A cost:** an approval costs one transaction fee. It does not require the owner to hold the approved amount.

The last property matters for what follows. An allowance is cheap to create, so the defense against fake endorsements cannot come from the edge itself.

4.2 Candidate mechanisms on Stacks

Table 1 compares the ERC-20 allowance with the Stacks mechanisms of Section 2.2. No single mechanism has all five properties. Post-conditions are explicit, signed by the asset owner and bounded per asset, but they last for one transaction and do not name a spender. Asset allowances in Clarity 4 name a bound but are written into code, so they express the developer’s decision rather than a user’s. Extension maps are standing and revocable but usually unbounded. Static references are permanent and say nothing about assets.

4.3 The choice

We split the role of the allowance in two.

Wallet to contract: post-conditions. When a user signs a transaction that calls contract c and bounds their own outflow of asset a by x , they let c , and every contract that c calls in that transaction, move up to x of a from their account. This is the meaning of $\text{approve}(c, x)$ on token a , restricted to one transaction. A post-condition does not name a spender, so we attribute the authorization to the contract named in the transaction’s payload, the entry point the user chose. On Ethereum the user approves the router, and the router moves the tokens into a pool; on Stacks the user calls the router, and the router’s calls move the tokens. In both cases the endorsement goes to the router, and the pool receives score from the router through a contract edge. To recover a standing state from per-transaction authorizations we apply EndorseRank’s overwrite rule to each wallet, contract and asset: only the most recent authorization counts (Section 5.2).

Contract to contract: references in code. A contract’s source states which contracts it calls,

Table 1: The ERC-20 allowance and the Stacks mechanisms that carry part of its meaning.

Mechanism	Granted by, to	Bound	Lifetime	Cost to create
ERC-20 approve / permit	Token owner, to a named spender	Amount per token	Until overwritten	One fee; no balance needed
Post-condition on the origin's outflow	Transaction origin, to the called contract and whatever it calls	Amount per asset, or a named NFT	One transaction	One fee
as-contract? allowance (Clarity 4)	Contract code, to the code in its body	Amount per asset per call; none with with-all- assets-unsafe	Permanent	One deployment
restrict- assets? (Clarity 4)	Contract code on behalf of a principal, to the code in its body	Amount per asset per call	Permanent	One deployment
Legacy as- contract	Contract code, to the code in its body	None	Permanent	One deployment
Extension or allow-list map	Contract governance, to another contract	Usually none	Until disabled	A governance decision
contract- call?, use- trait, impl- trait	Contract code, to the referenced contract	Not about assets	Permanent	One deployment

which traits it implements and uses, and which code it lets act with its own authority. We use two kinds of edges. A *delegation* edge $A \rightarrow B$ means that A calls B inside `as-contract?`, `legacy as-contract` or `restrict-assets?`, or that A 's governance has enabled B as an extension. In each case A 's developer or governance has put assets or authority at B 's disposal, which is the contract-level counterpart of an approval. A *dependency* edge $A \rightarrow B$ means that A calls B by static dispatch, or that A implements or uses a trait defined in B . These are the edges a package registry counts as imports.

What we leave out. Under dynamic dispatch the concrete callee is chosen by whoever calls A , not by A 's developer. If a user passes token X to router R , the choice of X is the user's, and the user's post-condition on X already records it as an authorization of R . Turning such a run-time

binding into an edge $R \rightarrow X$ would credit R with an endorsement its developer never made, and it would let anyone create edges out of a popular router by calling it. We therefore do not add edges for run-time trait bindings. Section 5.7 shows what happens when they are added.

4.4 Which post-conditions count

A post-condition counts as an authorization of the entry contract when all of the following hold:

- **Principal:** it constrains the transaction origin, either through the origin principal type or through a standard principal equal to the origin's address. Post-conditions on other principals do not authorize anything on the origin's behalf. A lower bound on what a pool contract must send, for example, is the user's guard against slippage.
- **Condition:** for STX and fungible tokens it is an upper bound ($=$, $<$ or $\leq$); for a non-fungible token it is *will send* or *may send*. A lower bound ($>$ or $\geq$) on the origin's own outflow leaves the amount open, and we treat it like an outflow in Allow mode.
- **Outcome:** the transaction succeeded. Failed transactions move no assets.

Transactions in Allow mode let uncovered assets leave the origin's account without limit. We count such outflows as *blanket authorizations* of the entry contract, the counterpart of an unlimited ERC-20 approval, which EndorseRank kept as a strong endorsement [11]. They are flagged so that a sensitivity analysis can drop them. Originator mode, available since Epoch 3.4, still requires the origin's outflows to be covered, so it fits the rules above unchanged.

4.5 AWP needs no replacement

AWP is built from transfers, and Stacks records every STX, fungible and non-fungible transfer as an event of the transaction that caused it [3]. AWP can therefore run on Stacks as published: an edge from sender to recipient for each transfer, weighted by value and by the logistic decay $\sigma(\Delta t) = 1/(1 + \exp(k(\Delta t - t_0)))$ with $k = 0.01$ and $t_0 = 180$ days [10]. Its graph contains wallets and contracts; we report the scores of contracts only. AWP serves as a baseline in Section 7.

5 PC-EndorseRank

5.1 Notation

Let T_0 be the freeze time of the ranking, C the set of contracts deployed on Stacks mainnet up to T_0 , and $\mathcal{W} = (T_0 - \Delta, T_0]$ an observation window, with $\Delta = 180$ days as in the ERC-20 study [11]. For a transaction τ in $\mathcal{W}$, $o(\tau)$ is its origin (a standard principal, called a wallet below), $c(\tau) \in C$ the contract named in its payload when the payload is a contract call, $f(\tau)$ its fee, and $\text{out}_a(\tau)$ the amount of asset a that $o(\tau)$ sent or burned during τ , read from the transaction's asset events. $\text{Auth}(\tau)$ is the set of assets for which τ carries an authorizing post-condition under the rules of Section 4.4, including blanket authorizations from Allow mode. $p(a)$ is the price of asset a at T_0 in a common unit; it is zero for assets without a reliable price and, in the primary analysis, for non-fungible tokens.

5.2 Authorization records and the latest-state rule

Definition 1 (Authorization record). For a wallet u , a contract c and an asset a , let $\tau^*(u, c, a)$ be the latest successful transaction in $\mathcal{W}$ with $o(\tau) = u$, $c(\tau) = c$ and $a \in \text{Auth}(\tau)$. Its value is $v(u, c, a) = p(a) \text{out}_a(\tau^*)$. The authorization of c by u is $A(u, c) = \sum_a v(u, c, a)$, the set of contracts u has authorized is $C_u = \{c : \tau^*(u, c, a) \text{ exists for some } a\}$, and the share of c in u 's endorsement is

$$s_u(c) = \begin{cases} A(u, c) / \sum_{c' \in C_u} A(u, c') & \text{if the denominator is positive,} \\ 1/|C_u| & \text{otherwise.} \end{cases} \quad (3)$$

Two choices differ from a literal copy of EndorseRank. First, the value is the amount that actually left the wallet, not the bound written in the post-condition. In a successful transaction an upper bound is never below the outflow, and a bound, like an ERC-20 allowance, can be set far above what the wallet holds at no cost; the realized outflow requires the asset to move. Second, the latest-state rule applies per wallet, contract and asset, as EndorseRank applies it per owner, spender and token. A wallet that calls the same contract a thousand times counts once,

with the value of its most recent call, so repeated calls from one wallet do not add weight.

5.3 Why the restart has to change

Moving EndorseRank to Stacks without further change would mean a graph with wallets and contracts as nodes, edges $u \rightarrow c$ weighted by $A(u, c)$, edges between contracts, and a uniform restart. The following proposition shows what such a graph measures.

Proposition 1 (Uniform restart counts wallets). *Let G have node set $W \cup C$ with N nodes, edges $u \rightarrow c$ from wallets to contracts with transition probabilities $s_u(c)$, edges among contracts, and the uniform restart $\pi_v = 1/N$. Then every wallet has the same score $r_W = (1 - d + d\delta)/N$, where δ is the score held by dangling nodes, and the score that wallets pass to a contract c is $d r_W \sum_u s_u(c)$. Adding k wallets that each authorize only a contract x adds $k d r_W$ to the score passed to x , whatever amounts they authorize.*

Proof. No edge ends at a wallet: contracts do not originate transactions, and edges between contracts end at contracts. By Equation (2) each wallet's score is therefore $(1 - d)/N + d\delta/N$, which does not depend on the wallet. A wallet u passes $d r_W$ along its out-edges in proportions $s_u(c)$, so the total passed to c is $d r_W \sum_u s_u(c)$. A wallet that authorizes only x has $s_u(x) = 1$. □

Under a uniform restart, the authorized amounts only decide how each wallet splits a fixed contribution among its contracts. The part of a contract's score that comes from wallets is a count of wallets in which each wallet's single vote is divided among the contracts it uses, and a Sybil wallet costs one transaction fee and brings a full vote. Value weighting does not change this. The proposition also suggests one reason why, in the ERC-20 study, raw in-degree counts matched held-out approvals better than EndorseRank [11]: an allowance graph in which few spenders are also owners is close to bipartite, and on such a graph the walk adds little to a count.

5.4 Wallet weights and the restart distribution

We therefore remove wallets from the node set and let them act only through the restart distribution, with a weight $\omega(u)$ that the attacker has to pay for:

$$\omega(u) = \sum_{\tau \in \mathcal{W}: o(\tau)=u} \min(f(\tau), f_{\max}), \quad \pi_c = \frac{\sum_u \omega(u) s_u(c)}{\sum_{u: C_u \neq \emptyset} \omega(u)}. \quad (4)$$

The weight of a wallet is the fees of all transactions it originated in the window, each capped at $f_{\max}$, the 99th percentile of fees in $\mathcal{W}$. Fees are counted for the origin even when a sponsor paid them, since sponsored fees are still a cost to the party that wants the endorsement. Equation (4) is the restart a walk would see if it started at a wallet chosen in proportion to ω and took one step along the authorization edges.

Proposition 2 (Splitting a fee budget gains nothing). *Suppose an attacker spends a fee budget F through wallets that authorize only x . Under Equation (4) their contribution to the numerator of π_x is at most F , however the budget is divided among wallets. If instead $\omega(u) = g(\text{fees of } u)$ for a concave g with $g(0) = 0$, dividing F equally among k wallets contributes $k g(F/k) \geq g(F)$; with $\omega \equiv 1$ it contributes k for a cost of k minimum fees.*

Proof. The first claim follows from linearity and the cap, which can only lower a counted fee.

A concave g with $g(0) = 0$ is subadditive, so $g(F) = g(\sum_{i=1}^k F/k) \leq \sum_{i=1}^k g(F/k)$. $\square$

Linear weights make influence proportional to money spent on transactions, which an attacker cannot avoid by creating identities. The price is that a wallet that transacts a lot counts more than one that transacts rarely; a one-wallet-one-vote rule would need an identity layer that Stacks does not have, and Cheng and Friedman’s result says that no symmetric rule escapes this [15]. Against $\omega \equiv 1$ an attacker buys one unit of influence with one minimum fee. Against Equation (4), matching the influence of n honest wallets costs what those wallets paid, so the cost rises by the ratio of the mean fee of an honest wallet to the minimum fee. Section 7 measures this ratio on mainnet. The sensitivity analysis also uses capital-days, the time-averaged STX balance of a wallet over $\mathcal{W}$: it is split-invariant like fees, and the same coins cannot sit in two wallets at once.

Stacks miners collect transaction fees, so a miner could pay fees to itself. The cap $f_{\max}$ limits

what a single self-paid transaction can buy; Section 8 returns to this.

5.5 Edges between contracts

Let E_{del} and E_{dep} be the delegation and dependency edges of Section 4. They are extracted from source and from governance transactions as follows.

- **Dependency:** $A \rightarrow B$ if A contains `contract-call?` with target B written as an absolute principal, as a relative principal `.name` (a contract of the same deployer), or as a constant bound to such a principal (Clarity 5); or if A has `impl-trait` or `use-trait` referring to a trait defined in B .
- **Delegation:** $A \rightarrow B$ if a resolved call from A to B lies in the body of `as-contract?`, legacy `as-contract` or `restrict-assets?`; or if A follows the `ExecutorDAO` pattern and the latest `set-extension` call in A 's history enables B .
- **Excluded:** calls whose target is a trait-typed argument, self-loops, and references to contracts that do not exist at T_0 .

An edge in both sets counts as delegation. Weights are $\beta_{\text{del}} = 2$ and $\beta_{\text{dep}} = 1$, fixed in advance with $\{1, 2, 4\}$ for β_{del} in the sensitivity analysis. Delegation weighs more because it puts assets or authority at the callee's disposal, which is the distinction between authorization and use on which `EndorseRank` rests. The transition matrix is $P_{AB} = \beta(A, B) / \sum_{B'} \beta(A, B')$.

5.6 Score

PC-`EndorseRank` is the solution of Equation (2) on the node set C with transition matrix P and restart π from Equation (4), with $d = 0.85$, an ℓ_1 tolerance of 10^{-8} and at most 300 iterations, the settings of the ERC-20 study [11]. Algorithm 1 lists the steps. Contracts with zero score have no evidence of use; the registry lists them but does not rank them.

Proposition 3 (Score of an attacker's contracts). *For a set $S \subseteq C$ let $\Phi_{\text{in}}(S) = \sum_{v \notin S} r_v P_{vS}$ be the score that flows into S along edges from contracts outside S , where $P_{vS} = \sum_{y \in S} P_{vy}$.*

Then

$$r(S) \leq \frac{\pi(S)}{1-d} + \frac{d}{1-d} \Phi_{\text{in}}(S). \quad (5)$$

In particular, if $\pi(S) = 0$ and no contract outside S has an edge into S , then $r(S) = 0$.

Proof. Summing Equation (2) over S gives $r(S) = (1-d)\pi(S) + d \sum_v r_v P_{vS} + d \delta \pi(S)$. Nodes of S with out-edges send at most all of their score into S and dangling nodes send none, so $\sum_{v \in S} r_v P_{vS} \leq r(S) - \delta_S$, where δ_S is the dangling score in S . Hence $(1-d)r(S) \leq (1-d)\pi(S) + d \Phi_{\text{in}}(S) + d(\delta \pi(S) - \delta_S)$. Since $\delta \leq 1$ and $\delta_S \geq 0$, the last term is at most $d \pi(S)$, which gives the bound. If $\pi(S) = 0$ and $\Phi_{\text{in}}(S) = 0$, the inequality reads $r(S) \leq 0$. $\square$

The two terms on the right are the two things an attacker must obtain. $\pi(S)$ is bought with fees on transactions that authorize contracts in S , at the price set by Proposition 2. $\Phi_{\text{in}}(S)$ requires contracts that already have score to reference S in their code or to enable it as an extension, which are decisions of other developers and of governance; these are the attack edges of the social-graph defenses [39, 18]. Deploying contracts that reference a target adds members to S but raises neither term. Under a uniform restart, by contrast, every deployed contract brings restart mass $1/|C|$ with it, so the bound grows with the number of contracts the attacker deploys.

Algorithm 1 PC-EndorseRank

Require: contracts C with source at T_0 ; transactions in $\mathcal{W}$; prices p ; parameters $d, \beta_{\text{del}}, \beta_{\text{dep}}, f_{\text{max}}$

Ensure: scores $\mathbf{r}$ on C

- 1: **for** each successful contract call $\tau \in \mathcal{W}$ and each $a \in \text{Auth}(\tau)$ **do**
 - 2: keep τ as $\tau^*(o(\tau), c(\tau), a)$ if it is later than the stored one
 - 3: **end for**
 - 4: $A(u, c) \leftarrow \sum_a p(a) \text{out}_a(\tau^*(u, c, a))$; compute $s_u(c)$ by Equation (3)
 - 5: $\omega(u) \leftarrow \sum_{\tau: o(\tau)=u} \min(f(\tau), f_{\text{max}})$; compute π by Equation (4)
 - 6: parse each contract; resolve literal, relative and constant targets; collect E_{dep} and E_{del} ; add enabled extensions to E_{del}
 - 7: $P_{AB} \leftarrow \beta(A, B) / \sum_{B'} \beta(A, B')$
 - 8: $\mathbf{r} \leftarrow \pi$
 - 9: **repeat**
 - 10: $\mathbf{r}' \leftarrow (1-d)\pi + dP^\top \mathbf{r} + d\delta(\mathbf{r})\pi$; swap $\mathbf{r}$ and $\mathbf{r}'$
 - 11: **until** $\|\mathbf{r} - \mathbf{r}'\|_1 < 10^{-8}$ or 300 iterations
 - 12: **return** $\mathbf{r}$
-

5.7 A synthetic illustration

To show the propositions at work we built a small synthetic graph (script in the paper’s repository; parameters in Appendix B). It has 14 honest contracts, among them a SIP-010 trait, six tokens, three pools that pay out tokens inside `as-contract?`, a router, a vault and a DAO, and 300 honest wallets that authorize one to three entry contracts and pay on average 31.1 times the minimum fee over the window. An attacker deploys a token x and tries to lift it into the top 3 by three attacks: A1 deploys contracts that each call x (cost 10 fee units per contract); A2 creates wallets that each authorize x once at the minimum fee; A3 calls the honest router with x as its trait argument. Table 2 reports the smallest cost that reaches the top 3 under five scoring rules.

Table 2: Smallest attack cost, in units of the minimum call fee, that lifts the attacker’s token into the top 3 of a synthetic graph with 14 honest contracts. “Not reached” means the token stays outside the top 3 for every attack size up to 200,000. The graph is made up; the table illustrates Propositions 1–3 and is not a result about Stacks mainnet.

Scoring rule	A1	A2	A3
Distinct callers	not reached	59	not reached
Dependency PageRank, uniform restart	10	not reached	not reached
EndorseRank on Stacks, uniform restart	750	75	not reached
PC-EndorseRank (fee-weighted restart)	not reached	2,234	not reached
PC-EndorseRank with runtime trait bindings	not reached	2,234	3,952

Each baseline fails against one attack. Counting callers falls to the wallet farm (A2) at 59 units. Dependency PageRank with a uniform restart, the `teaRank` pattern, falls to a single farm contract (A1, 10 units) because each deployed contract brings its own restart mass. EndorseRank with a uniform restart falls to both: 75 wallets suffice, close to the count of callers as Proposition 1 predicts, and farm contracts work too because they are nodes with restart mass. PC-EndorseRank is not moved by A1 at any size, as Proposition 3 requires, and A2 costs 2,234 units, about 30 times the cost under the uniform restart and close to the mean fee of an honest wallet (31.1 units). A3 moves the token only when run-time trait bindings are added as router edges. In this graph that attack costs more than A2, because honest calls already bind the router’s tokens hundreds of times; a router whose score comes mostly from delegation rather than from calls would be cheaper to capture.

5.8 Families, packages and search

Clone families. Two contracts belong to the same family when their normalized sources have the same hash. Normalization removes comments and whitespace and replaces string and numeric literals, literal principals and the names of defined tokens, maps and variables with placeholders, then hashes the token stream. Contracts with identical code bodies also share the value of `contract-hash?`. Near-duplicates found by MinHash [51] with a Jaccard similarity of at least 0.9 are shown as related but not merged. A family is represented by its member with the highest score, earliest deployment breaking ties, and the family is ranked by that member’s score. Using the maximum rather than the sum means that a thousand copies of a contract add nothing to its family.

Packages. A package is a lineage of contracts declared by its publisher (Section 6). It is ranked by the score of its latest version, with older versions listed under it.

Search. Following Equation (1), a contract c matching query q receives

$$\text{score}(q, c) = \text{BM25}(q, c) \cdot (\ln(e + |C| r_c))^\alpha, \quad \alpha = 1, \quad (6)$$

where BM25 [56] runs over the contract name, the publisher scope, function and trait names from the ABI, comments in the source and the package README, and $|C| r_c$ equals one for a contract of average score. Results can be filtered by trait conformance (SIP-009, SIP-010, SIP-013), Clarity version and network.

6 The registry and cpm

6.1 Architecture

Figure 1 shows the components. An indexer fills a PostgreSQL database with contracts, transactions, post-conditions and asset events. For the history it reads a self-hosted Stacks node and Stacks Blockchain API rather than the hosted API, because a full backfill through the hosted service is slow under its rate limits and because a pinned archive makes the evaluation reproducible.

New blocks arrive through Chainhooks or the node’s event interface [3, 52]. An analyzer parses each contract, extracts the references and allowances of Section 5, computes the normalized hash and runs static checks. A daily job builds the graph, computes PC-EndorseRank and the clone families, and refreshes the search index. The web front end and the `cpm` client read from the same search API, and `cpm` checks what it receives against the chain.

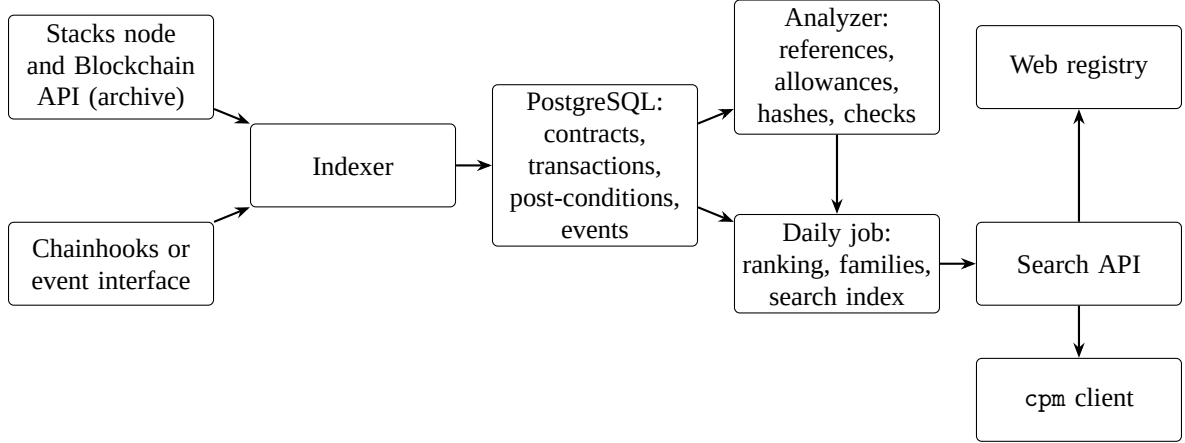


Figure 1: Components of the Clarity Registry. The `cpm` client also reads code hashes directly from a Stacks node to check what the API returns.

6.2 Package model

Scopes. Every contract already has a publisher: the address in its principal. The registry therefore indexes every contract under its principal, and a publisher may claim a readable scope in two ways. The address itself is always a valid scope. A BNS name such as `alice.btc` becomes a scope if it resolved to the deployer’s address when the claim was made. BNS names are SIP-009 tokens and can be sold [57], so the registry records the name, its owner and the block height of the claim. If the name later changes hands, the scope is frozen: existing versions keep their identity, new versions need a new claim, and `cpm` warns when a scope’s owner differs from the one recorded in the project’s lockfile. This guards against the takeover of a trusted name, an attack known from other registries [30].

Versions. A deployed contract cannot change, so each version of a package is a separate contract, and `token-v1` and `token-v2` are unrelated on chain. The publisher states the relation in a manifest: the package name, the semantic version of each contract, the predecessor of each version, the matching testnet principal, a license identifier, a repository URL, a README

and links to audit reports. The registry accepts a manifest only if every listed contract was deployed by the scope’s address, and only if the manifest is signed by that address with a SIP-018 structured-data signature [58] or submitted in a transaction from it. No other key infrastructure is needed, because the address that signs is the address that deployed.

Unclaimed contracts. Contracts without a manifest are still indexed, under their principal, with trait labels, a clone family and a score, but without versions or a README.

6.3 Commands

cpm wraps Clarinet rather than replacing it. Table 3 lists its commands. The central distinction is between referencing and copying. On npm, installing a package copies code into the project. A Clarity contract that a project wants to *use* is already deployed, shared by everyone, and called by its principal; what a project adds is a reference. `cpm add` records that reference as a Clarinet requirement, so that local tests can call a copy that Clarinet deploys on its local networks [5], while a mainnet deployment deploys nothing for it. A project that wants its own instance, a token built from a SIP-010 template for example, *copies* the source with `cpm create` or `cpm fork` and deploys the copy under its own address. Copying is allowed only when the package’s license permits it; publication on chain does not make code open source.

Table 3: Commands of the cpm client.

Command	Effect
<code>cpm search <query></code>	Searches with Equation (6); filters for trait, Clarity version and network.
<code>cpm info <pkg></code>	Shows versions, principals, family, score and the signals of Table 4.
<code>cpm add <pkg>[@ver]</code>	References a deployed contract: resolves it and its static dependencies, checks Clarity version and epoch against the project, adds Clarinet requirements and writes the lockfile. <code>cpm install</code> is an alias.
<code>cpm create <template> <name></code>	Copies a template’s source into <code>contracts/</code> , renames it, fills its declared parameters and records the template’s family hash. Refused if the license does not allow copying.
<code>cpm fork <pkg></code>	As <code>create</code> , for any package whose license allows it.
<code>cpm why <contract></code>	Prints the dependency paths from the project’s contracts to the given contract.
<code>cpm audit</code>	Reports the signals of Table 4 for every dependency.
<code>cpm verify</code>	Reads the code hash of every locked contract from a node and compares it with the lockfile.

6.4 Lockfile

Version ranges, the main subject of npm lockfiles, do not exist for deployed contracts. The `cpm` lockfile records identities instead: for each dependency, its mainnet principal and testnet counterpart, its code hash as returned by `contract-hash?`, its deployment height and Clarity version, its family hash, its license, the scope owner seen when it was added, and whether it was referenced or copied. `cpm verify` reads the code hashes from a node, through a read-only call to a contract that returns `contract-hash?` such as the registry contract below, and compares them with the lockfile, so that a compromised API or mirror cannot substitute different source for a contract the project depends on.

6.5 An optional registry contract

Names can also live on chain. A small registry contract offers `publish(name, version, contract)`, which checks with `principal-destruct?` that the address part of `contract` equals `tx-sender`, stores the result of `contract-hash?` for that contract next to the name and version, and prints an event for indexers. It also offers a read-only function that returns `contract-hash?` for any principal, which `cpm` uses for verification. The registry works without this contract; with it, names and hashes are recorded on chain and do not depend on the operator of the web service.

6.6 Signals shown to users

A ranking orders results; it does not say that a contract is safe. Each contract page and `cpm audit` show the signals of Table 4 next to the score, and the registry keeps indexing separate from recommending. Every contract is indexed. A “recommended” label, if offered, requires criteria beyond the score, such as an audit link, a minimum age and no high-severity findings, and the registry does not call a contract malicious on the strength of its score.

Table 4: Signals shown with each contract.

Signal	Why it matters
with-all-assets-unsafe, or legacy <code>as-contract</code> around calls to other contracts	The contract lets other code move its assets without a limit; STACY flags calls inside <code>as-contract</code> [53].
Trait-typed argument called inside <code>as-contract</code> ?	The contract’s assets are exposed to code chosen by its caller.
Share of calls in Allow mode	Users of the contract are signing transactions without post-condition protection.
Clone family and representative	Shows whether the contract is the most used member of its family or one of many copies.
Age, Clarity version, epoch	Older Clarity versions lack asset allowances; very new contracts have little history.
Static analysis findings	Output of STACY and the registry’s own checks.
Audit links	Taken from the manifest; the registry does not verify the audit itself.

7 Evaluation design

The evaluation is fixed before its label window opens, following the registered replication of the ERC-20 study [11]. The configuration file that holds the dates, parameters, random seeds and decision rules below is committed before 1 November 2026, and its hash is reported with the results.

7.1 Research questions

- **RQ1 (signal):** How often do Stacks transactions carry authorizing post-conditions, and how often do contracts use asset allowances, legacy `as-contract` and extension maps?
- **RQ2 (attacker cost):** How much must an attacker pay to lift a contract into the top of each ranking?
- **RQ3 (adoption):** Does PC-EndorseRank at the freeze time agree with later adoption as well as the best baseline does, and do the edges between contracts add information beyond the restart distribution?
- **RQ4 (noise):** What share of deployed contracts are copies or show no use, and how much does grouping by family change search results?

7.2 Data and windows

All data come from Stacks mainnet through a self-hosted node and Stacks Blockchain API. The ranking is frozen at $T_0 = 31$ October 2026, 23:59:59 UTC. The contract graph uses every contract deployed up to T_0 . Authorizations and fees use the 180 days before T_0 , which begin after the planned Epoch 3.4 activation at Bitcoin block 943,333, so that every transaction in the window follows the same post-condition rules [20, 24]; if activation came later, the window starts at activation and is shorter. Labels come from the window L from 1 November 2026 to 31 January 2027. A replication repeats everything with $T'_0 = 31$ January 2027 and labels from 1 February to 30 April 2027. Prices are taken once, at T_0 , from a public market source named in the configuration file.

7.3 Methods compared

Table 5 lists the methods. Raw counts are included because in the ERC-20 study they matched held-out approvals better than both PageRank variants [11], and a new ranking should be shown to beat them or to be no worse while costing an attacker more.

Table 5: Methods compared. All PageRank variants use $d = 0.85$, an ℓ_1 tolerance of 10^{-8} and at most 300 iterations.

Method	Definition
PC-EndorseRank	Section 5: contracts as nodes, delegation and dependency edges, restart from fee-weighted authorizations.
B1 Callers	Number of distinct wallets with an authorizing call to the contract in the window.
B2 Dependents	Number of distinct contracts from other deployers with an edge to the contract, the analogue of pkg.go.dev’s importer count [6].
B3 Dependency PageRank	Contracts as nodes, all references with weight one, uniform restart (the teaRank pattern).
B4 AWP	Transfer graph of wallets and contracts with logistic decay ($k = 0.01$, $t_0 = 180$ days), uniform restart [10]; contract scores reported.
B5 EndorseRank on Stacks	Wallets and contracts as nodes, edges $u \rightarrow c$ weighted by $A(u, c)$ plus contract edges, uniform restart.
Ablations	Score equal to π alone (no contract edges); run-time trait bindings added as edges; Allow-mode authorizations dropped; $\omega \equiv 1$; ω from capital-days; raw token units instead of prices; $\beta_{\text{del}} \in \{1, 4\}$.

7.4 Labels for adoption

No ground truth says which contracts matter. As in the ERC-20 study, we use observable outcomes as labels and report rank agreement with each.

- **Y1 New callers:** wallets whose first authorizing call to the contract falls in L . A filtered version counts only wallets that originated transactions on at least three different days in L .
- **Y2 New dependents:** contracts deployed in L by other deployers with a delegation or dependency edge to the contract.
- **Y3 Developer adoption:** public GitHub repositories whose `Clarinet.toml` lists the contract as a requirement and that first do so in L , from code-search snapshots at T_0 and at the end of L . Used in the hypothesis test only if at least 100 contracts have a non-zero value.
- **Y4 Curated set:** contracts of protocols listed for Stacks by DefiLlama at T_0 [59] or covered by a public audit report dated before T_0 , collected by a rule written in the configuration file before scores are computed.

Y1 is close to B1 by construction, as the transfer checks of the ERC-20 study were close to AWP; agreement between them shows consistency rather than independent predictive power. Rank agreement is computed over the evidence set, the contracts with at least one authorization in the window or at least one incoming edge at T_0 , using Kendall's τ_b [60] and Spearman's ρ [61]. We also report NDCG@100 with Y1 and Y2 as graded gains [62], and precision at 100 and the area under the ROC curve for Y4.

7.5 Attack simulation

Attacks are injected into a copy of the frozen data, and every method is recomputed on the modified copy.

- **A1 Contract farm:** k new contracts, each calling the target by static dispatch.
- **A2 Wallet farm:** k new wallets, each sending one authorizing call to the target at the lowest fee that was included in the window; for AWP each call also transfers the smallest

priced amount to the target.

- **A3 Wash flow:** one wallet sends value to the target and receives it back m times, the target being under the attacker’s control.
- **A4 Binding injection:** m calls to the highest-scored router that takes a SIP-010 trait argument, with the target as that argument; relevant only to the ablation with run-time bindings.

Costs are measured in STX: a deployment costs the median fee of contract deployments under 2 KB in the window, a call costs the 5th percentile of included contract-call fees, and capital that returns to the attacker is not counted. For each of 200 targets drawn at random from evidence-set contracts that lie in the lower half of every method’s ranking, we find by doubling and bisection the smallest k or m that lifts the target into the top 100 overall (the top 5% if the evidence set has fewer than 2,000 contracts) and into the top 10 of its trait category. A method that the attack cannot move within a budget of 100,000 STX is assigned that budget, which understates its advantage.

7.6 Hypotheses and decision rules

- **H1 (attacker cost):** the median over targets of the log ratio between the cost under PC-EndorseRank and the cost under a baseline is positive, with the lower bound of its 95% bootstrap confidence interval above zero, for every pair in which the attack moves the baseline: A1 against B2, B3 and B5, and A2 against B1, B4 and B5. A3 against B4 is reported without a test.
- **H2 (adoption, non-inferiority):** the mean of τ_b over Y1 and Y2 (and Y3 if eligible) for PC-EndorseRank is at least that of the best baseline minus 0.02, judged by the lower bound of a paired 95% bootstrap interval of the difference. The margin is the one used in the ERC-20 replication [11].
- **H3 (propagation):** τ_b with Y2 is higher for PC-EndorseRank than for the ablation that scores contracts by π alone, with the lower bound of the paired interval above zero.

The paper will claim that PC-EndorseRank raises the cost of manipulation without losing agreement with adoption only if H1 and H2 both hold in the main window. H3 is secondary. All

coefficients are reported whatever their direction, together with the replication and the sensitivity analyses. Intervals use 2,000 bootstrap resamples over contracts for agreement and over targets for costs, with percentile limits [63].

7.7 Descriptive measures

For RQ1 we report the share of contract calls in the window by post-condition mode; the share with at least one authorizing post-condition on the origin; the ratio between declared bounds and realized outflows; for each Clarity version, the share of contracts that use `as-contract?`, `restrict-assets?`, `with-all-assets-unsafe`, or legacy `as-contract` around external calls; the number of `ExecutorDAO`-style contracts and enabled extensions; and the share of calls that pass a trait-typed argument. For RQ4 we report the share of contracts with no evidence of use, the distribution of family sizes, the largest families, and, for 50 queries drawn from frequent function and trait names, the share of top-10 results that are not their family’s representative with and without grouping, and how often a representative is the earliest deployment of its family. We also report graph sizes and the running time of each method.

7.8 Checks of `cpm`

On a Clarinet devnet we check three cases: `cpm add` of a SIP-010 token package and of a pool that depends on it produces requirements under which the project’s tests pass; `cpm create` from a template produces a contract that deploys; and `cpm verify` detects a contract whose source has been altered by a mock API.

8 Discussion and limitations

Who writes post-conditions. Users rarely write post-conditions themselves; the application’s front end builds them and the wallet displays them for approval. An ERC-20 approval is built the same way, often for an unlimited amount, so post-conditions are no weaker as evidence of intent than the allowances `EndorseRank` used. A malicious front end could set loose bounds,

but PC-EndorseRank values an authorization by the realized outflow rather than by the bound, and a front end only shapes the transactions sent to its own contracts. If Allow mode turns out to be common (RQ1), much of the signal will consist of blanket authorizations. The sensitivity analysis that drops them shows how much the ranking depends on them.

Fees as identity. Weighting wallets by fees makes the ranking favor contracts used by active wallets, and heavy users count more than occasional ones. We accept this because the alternative, one vote per wallet, is what Sybil wallets exploit (Proposition 1). Two weaknesses remain. Stacks miners collect fees, so a miner can pay fees to itself; the cap $f_{\max}$ bounds what each such transaction can buy but does not remove the incentive. Fees on Stacks are low, so the absolute cost of an attack on a sparsely used category may be small even when it is many times the cost under a uniform restart. Capital-days, the alternative weight, cannot be recycled across wallets at the same moment but favor wealthy wallets even more.

Prices. Valuing outflows needs prices. Illiquid tokens get no price and therefore no influence over how a wallet splits its endorsement; tokens with manipulated prices could gain influence, which is why prices are taken from one public source at one time. The ablation with raw token units, the convention of the ERC-20 study, shows how much the prices matter.

Run-time bindings. Excluding run-time trait bindings protects popular routers from being used as amplifiers, but it means that a token traded only through routers receives score only from the users who authorize it directly and from pools that name it in code. Tokens are among the most searched contracts, so the evaluation reports the ablation with bindings added.

Static extraction. The parser resolves literal, relative and constant targets. Principals stored in data variables or maps at run time are not resolved, and allow-lists other than the ExecutorDAO pattern are not read. Such references are missing edges, which lower the scores of the contracts they point to. Extracting state-based references from governance transactions is future work.

Families. Normalization can merge contracts that share code but differ in parameters that matter, such as a fee rate, and it cannot detect a copy that has been reformatted with renamed functions. Choosing the most used member as a family’s representative can place a popular copy above the original author; the registry therefore shows each family’s deployment history, with the earliest member marked.

Data access and reproducibility. The hosted Stacks Blockchain API limits request rates, and some research environments block it, so the evaluation uses a self-hosted node and API and publishes extraction scripts and manifests. Transaction-level data contain wallet addresses; they are public, but results are reported only in aggregate, wallets are never ranked or displayed, and the registry shows counts rather than lists of users.

Harm to new projects. Any ranking based on use is slow to notice new contracts. The registry lists unused contracts and offers a view sorted by deployment date, and it does not attach negative labels to a contract because its score is low.

Relation to the ERC-20 study. Proposition 1 offers one explanation for a result of the ERC-20 study, where raw in-degree counts matched held-out approvals better than EndorseRank [11]: on a nearly bipartite graph with a uniform restart, the walk adds little beyond a count. PC-EndorseRank differs in what it is meant to gain. Its claim is not better agreement with adoption, which H2 only requires to be no worse, but a higher price for manipulation, which H1 tests.

9 Conclusion

Clarity contracts are published as source, so a registry for them can be built from chain data alone, but it needs a ranking that resists fake popularity. EndorseRank cannot be moved to Stacks unchanged because Stacks has no allowance. The allowance relation splits into two parts: post-conditions record which contracts a wallet let move its assets, and delegation and dependency references in code record which contracts a contract relies on. AWP can be applied to Stacks transfers as it stands.

Moving wallets out of the graph and into the restart distribution is what makes the ranking resist Sybil attacks. With a uniform restart, a wallet–contract graph turns PageRank into a count of wallets, and dependency PageRank turns it into a count of deployed contracts; both are cheap to inflate. With a restart weighted by fees, an attacker pays for influence in proportion to what honest wallets have paid, and contracts that nobody authorizes and no honest contract references receive no score at all. The registry built on this ranking separates referencing a contract from copying it, ties scopes and versions to deployer addresses, groups clones by hash and verifies

code hashes on chain.

The evaluation set out in Section 7 will show whether the higher cost of manipulation holds on Stacks mainnet and whether it comes without a loss of agreement with later adoption. Both outcomes are reported whatever their direction.

References

- [1] Stacks Improvement Proposals. (n.d.). *SIP-002: Smart contract language*. <https://github.com/stacksgov/sips/tree/main/sips/sip-002>
- [2] Hiro Systems. (2024, August 23). *There's nowhere to hide with Clarity* [Blog post]. <https://www.hiro.so/blog/theres-nowhere-to-hide-with-clarity>
- [3] Hiro Systems. (n.d.). *Stacks Blockchain API reference*. <https://docs.hiro.so/apis/stacks-blockchain-api>
- [4] boomcrypto. (n.d.). *clarity-deployed-contracts* [GitHub repository]. <https://github.com/boomcrypto/clarity-deployed-contracts>
- [5] Stacks Documentation. (n.d.). *Clarinet: Contract deployment and requirements*. <https://docs.stacks.co/clarinet/contract-deployment>
- [6] The Go Authors. (n.d.). *pkgsite: internal/postgres/search.go* [Source code]. <https://github.com/golang/pkgsite/blob/master/internal/postgres/search.go>
- [7] Sonatype. (2024, April). *Devs flood npm with 15,000 packages to reward themselves with tea 'tokens'* [Blog post]. <https://www.sonatype.com/blog/devs-flood-npm-with-10000-packages-to-reward-themselves-with-tea-tokens>
- [8] Tea Inc. (2022, May 19). *Tea releases whitepaper outlining decentralized protocol for rewarding open source developers* [Press release]. Business Wire. <https://www.businesswire.com/news/home/20220519005165/en/>
- [9] Amazon Web Services. (2025, November 13). *Amazon Inspector detects over 150,000 malicious packages linked to token farming campaign*. AWS Security Blog. <https://aws.amazon.com/blogs/security/amazon-inspector-detects-over-150000-malicious-packages-linked-to-token-farming-campaign/>
- [10] Do, T., Do, D., & Nguyen, L. (2023). On-chain reputation ranking by Adaptive Weighted PageRank. In *2023 RIVF International Conference on Computing and Communication Technologies (RIVF)* (pp. 318–323). IEEE. doi:10.1109/RIVF60135.2023.10471809
- [11] Kwon, T. (2026). *Integrating ERC-20 allowance edges into PageRank for enhanced on-chain wallet reputation scoring* [Doctoral thesis in preparation]. University of South Africa.
- [12] Stacks Improvement Proposals. (n.d.). *SIP-010: Standard trait definition for fungible tokens*. <https://github.com/stacksgov/sips/tree/main/sips/sip-010>

- [13] Stacks Improvement Proposals. (n.d.). *SIP-005: Blocks, transactions, and accounts*. <https://github.com/stacksgov/sips/tree/main/sips/sip-005>
- [14] Stacks Improvement Proposals. (2025). *SIP-033: Clarity 4*. <https://github.com/stacksgov/sips/tree/main/sips/sip-033>
- [15] Cheng, A., & Friedman, E. (2005). Sybilproof reputation mechanisms. In *Proceedings of the 2005 ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems (P2PECON '05)* (pp. 128–132). ACM.
- [16] Gyöngyi, Z., & Garcia-Molina, H. (2005). Link spam alliances. In *Proceedings of the 31st International Conference on Very Large Data Bases* (pp. 517–528). VLDB Endowment.
- [17] Gyöngyi, Z., Garcia-Molina, H., & Pedersen, J. (2004). Combating web spam with TrustRank. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases* (pp. 576–587). VLDB Endowment.
- [18] Cao, Q., Sirivianos, M., Yang, X., & Pregueiro, T. (2012). Aiding the detection of fake accounts in large scale social online services. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)* (pp. 197–210). USENIX Association.
- [19] Ali, M., et al. (n.d.). *Stacks: A Bitcoin layer for smart contracts* [Whitepaper]. <https://stacks-network.github.io/stacks/stacks.pdf>
- [20] Stacks Improvement Proposals. (2026). *SIP-039: Clarity 5: Fixing known issues in Clarity*. <https://github.com/stacksgov/sips/tree/main/sips/sip-039>
- [21] Stacks Improvement Proposals. (n.d.). *SIP-009: Standard trait definition for non-fungible tokens*. <https://github.com/stacksgov/sips/tree/main/sips/sip-009>
- [22] Stacks Improvement Proposals. (n.d.). *SIP-013: Standard trait definition for semi-fungible tokens*. <https://github.com/stacksgov/sips/tree/main/sips/sip-013>
- [23] Hiro Systems. (n.d.). *Post-conditions* [Stacks.js documentation]. <https://docs.hiro.so/reference/stacks.js/post-conditions>
- [24] Stacks Improvement Proposals. (2026). *SIP-040: Improved post-conditions*. <https://github.com/stacksgov/sips/tree/main/sips/sip-040>
- [25] Stacks Documentation. (n.d.). *ExecutorDAO framework*. <https://docs.stacks.co/cookbook/clarity/example-contracts/executordao-framework>
- [26] Kikas, R., Gousios, G., Dumas, M., & Pfahl, D. (2017). Structure and evolution of package dependency networks. In *Proceedings of the 14th International Conference on Mining Software Repositories* (pp. 102–112). IEEE.
- [27] Decan, A., Mens, T., & Grosjean, P. (2019). An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1), 381–416.
- [28] Mujahid, S., Costa, D. E., Abdalkareem, R., Shihab, E., Saied, M. A., & Adams, B. (2021). Towards using package centrality trend to identify packages in decline. *IEEE Transactions on Engineering Management*. doi:10.1109/TEM.2021.3122012

- [29] Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber’s knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)* (pp. 23–43). Springer.
- [30] Ladisa, P., Plate, H., Martinez, M., & Barais, O. (2023). SoK: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy* (pp. 1509–1526). IEEE.
- [31] Vu, D.-L., Pashchenko, I., Massacci, F., Plate, H., & Sabetta, A. (2020). Typosquatting and com-bosquatting attacks on the Python ecosystem. In *2020 IEEE European Symposium on Security and Privacy Workshops* (pp. 509–514). IEEE.
- [32] Zimmermann, M., Staicu, C.-A., Tenny, C., & Pradel, M. (2019). Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security Symposium* (pp. 995–1010). USENIX Association.
- [33] Page, L., Brin, S., Motwani, R., & Winograd, T. (1998). *The PageRank citation ranking: Bringing order to the web* (Technical Report). Stanford Digital Library Technologies Project.
- [34] Langville, A. N., & Meyer, C. D. (2006). *Google’s PageRank and beyond: The science of search engine rankings*. Princeton University Press.
- [35] Haveliwala, T. H. (2003). Topic-sensitive PageRank: A context-sensitive ranking algorithm for web search. *IEEE Transactions on Knowledge and Data Engineering*, 15(4), 784–796. doi:10.1109/TKDE.2003.1208999
- [36] Jeh, G., & Widom, J. (2003). Scaling personalized web search. In *Proceedings of the 12th International Conference on World Wide Web* (pp. 271–279). ACM. doi:10.1145/775152.775191
- [37] Kamvar, S. D., Schlosser, M. T., & Garcia-Molina, H. (2003). The EigenTrust algorithm for reputation management in P2P networks. In *Proceedings of the 12th International Conference on World Wide Web* (pp. 640–651). ACM. doi:10.1145/775152.775242
- [38] Douceur, J. R. (2002). The Sybil attack. In *Peer-to-Peer Systems* (pp. 251–260). Springer. doi:10.1007/3-540-45748-8_24
- [39] Yu, H., Kaminsky, M., Gibbons, P. B., & Flaxman, A. (2006). SybilGuard: Defending against Sybil attacks via social networks. In *Proceedings of the 2006 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM ’06)* (pp. 267–278). ACM.
- [40] Bianchini, M., Gori, M., & Scarselli, F. (2005). Inside PageRank. *ACM Transactions on Internet Technology*, 5(1), 92–128.
- [41] Andersen, R., Borgs, C., Chayes, J., Hopcroft, J., Jain, K., Mirrokni, V., & Teng, S.-H. (2008). Robust PageRank and locally computable spam detection features. In *Proceedings of the 4th International Workshop on Adversarial Information Retrieval on the Web* (pp. 69–76). ACM. doi:10.1145/1451983.1452000
- [42] Do, H., & Do, T. (2023). PageRank and HodgeRank on Ethereum transactions: A measure for social credit. *International Journal of Software Innovation*, 11(1), 1–13. doi:10.4018/IJSI.315737

- [43] Lin, D., Wu, J., Fu, Q., Zheng, Z., & Chen, T. (2025). RiskProp: Account risk rating on Ethereum via de-anonymous score and network propagation. *IEEE Transactions on Dependable and Secure Computing*, 22(3), 2132–2145. doi:10.1109/TDSC.2024.3475825
- [44] Chen, T., et al. (2018). Understanding Ethereum via graph analysis. In *IEEE INFOCOM 2018: IEEE Conference on Computer Communications*. IEEE.
- [45] Victor, F. (2020). Address clustering heuristics for Ethereum. In *Financial Cryptography and Data Security (FC 2020)* (pp. 617–633). Springer. doi:10.1007/978-3-030-51280-4_33
- [46] Newman, Z., Meyers, J. S., & Torres-Arias, S. (2022). Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2353–2367). ACM.
- [47] Kondo, M., Oliva, G. A., Jiang, Z. M., Hassan, A. E., & Mizuno, O. (2020). Code cloning in smart contracts: A case study on verified contracts from the Ethereum blockchain platform. *Empirical Software Engineering*, 25(6), 4617–4675.
- [48] He, N., Wu, L., Wang, H., Guo, Y., & Jiang, X. (2020). Characterizing code clones in the Ethereum smart contract ecosystem. In *Financial Cryptography and Data Security (FC 2020)* (pp. 654–675). Springer.
- [49] Xia, P., Wang, H., Gao, B., Su, W., Yu, Z., Luo, X., Zhang, C., Xiao, X., & Xu, G. (2021). Trade or trick? Detecting and characterizing scam tokens on Uniswap decentralized exchange. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 5(3), 1–26. doi:10.1145/3491051
- [50] Roy, C. K., Cordy, J. R., & Koschke, R. (2009). Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming*, 74(7), 470–495.
- [51] Broder, A. Z. (1997). On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences 1997* (pp. 21–29). IEEE.
- [52] Stacks. (2026, February 5). *Chainhooks v2 is now generally available* [Blog post]. <https://www.stacks.co/blog/chainhooks-v2-is-now-generally-available>
- [53] CoinFabrik. (n.d.). *STACY: Static analyzer for Clarity* [Software]. <https://pypi.org/project/stacy-analyzer>
- [54] Ethereum Improvement Proposals. (2015). *ERC-20 token standard (EIP-20)*. <https://eips.ethereum.org/EIPS/eip-20>
- [55] Ethereum Improvement Proposals. (2020). *ERC-20 permit extension (EIP-2612)*. <https://eips.ethereum.org/EIPS/eip-2612>
- [56] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- [57] Stacks Documentation. (n.d.). *How BNSv2 works*. <https://docs.stacks.co/learn/network-fundamentals/bitcoin-name-system/architecture>
- [58] Stacks Improvement Proposals. (n.d.). *SIP-018: Signed structured data*. <https://github.com/stacksgov/sips/tree/main/sips/sip-018>

- [59] DefiLlama. (n.d.). *Stacks: Total value locked by protocol*. <https://defillama.com/chain/Stacks>
- [60] Kendall, M. G. (1938). A new measure of rank correlation. *Biometrika*, 30(1/2), 81–93. doi:10.2307/2332226
- [61] Spearman, C. (1904). The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1), 72–101. doi:10.2307/1412159
- [62] Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4), 422–446.
- [63] Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1–26. doi:10.1214/aos/1176344552

A Extraction rules

A.1 Post-conditions

Table 6 maps the post-condition fields of a transaction, as the Stacks Blockchain API returns them [3], to the rules of Section 4.4. Field names follow the API schema at the time of writing; the evaluation pins one API version in its configuration file and checks there how Originator mode and the *may send* condition of SIP-040 are encoded.

A.2 Clarity constructs

Table 7 lists the constructs the analyzer reads and the edges they produce.

B The synthetic example

The example of Section 5.7 is produced by `scripts/toy_sybil_example.py` with a fixed random seed. Its 14 honest contracts are a SIP-010 trait, six tokens that implement it, a math library, three pools that each use the trait and the library and pay out their two tokens inside `as-contract?`, a router that calls the three pools, a vault that pays out two tokens and uses the library, and a DAO that has enabled the vault and the router as extensions. Each of the

Table 6: Post-condition fields and how the extraction uses them.

Field	Use
<code>tx_status</code>	Only successful transactions are used.
<code>tx_type</code> , <code>contract_call</code> , <code>contract_id</code>	Contract calls only; the contract named here is the entry contract $c(\tau)$.
<code>sender_address</code> , <code>sponsored</code>	The origin $o(\tau)$; fees count for the origin whether or not a sponsor paid them.
<code>fee_rate</code>	The fee $f(\tau)$, capped at $f_{\max}$.
<code>post_condition_mode</code>	Deny and Originator: only covered outflows of the origin count as explicit authorizations. Allow: uncovered outflows of the origin count as blanket authorizations and are flagged.
<code>post_conditions[]</code> . <code>principal.type_id</code>	<code>principal_origin</code> , or <code>principal_standard</code> with the origin's address: the condition can authorize. Contract principals: never an authorization.
<code>post_conditions[].type</code>	<code>stx</code> , <code>fungible</code> or <code>non_fungible</code> ; with <code>asset</code> , identifies the asset a .
<code>post_conditions[].condition_code</code>	Upper bounds (<code>sent_equal_to</code> , <code>sent_less_than</code> , <code>sent_less_than_or_equal_to</code>) and NFT <code>sent</code> or <i>may send</i> : explicit authorization. Lower bounds on the origin: blanket authorization. <code>not_sent</code> : none.
<code>events[]</code> (asset transfers and burns)	The realized outflow $\text{out}_a(\tau)$: amounts sent or burned by the origin.

Table 7: Clarity constructs and the edges they produce for a contract A .

Construct in A	Edge
<code>(contract-call? 'SP...b f ...)</code>	Dependency $A \rightarrow b$
<code>(contract-call? .b f ...)</code>	Dependency $A \rightarrow b$ of the same deployer
<code>(define-constant T 'SP...b)</code> with <code>(contract-call? T f ...)</code> (Clarity 5)	Dependency $A \rightarrow b$
<code>(impl-trait 'SP...d.t)</code> , <code>(use-trait x 'SP...d.t)</code>	Dependency $A \rightarrow d$, the contract defining trait t
Any resolved call to b inside <code>as-contract?</code> , <code>as-contract</code> or <code>restrict-assets?</code>	Delegation $A \rightarrow b$
<code>set-extension</code> in an ExecutorDAO-style A , latest call enabling b	Delegation $A \rightarrow b$
<code>(contract-call? x f ...)</code> with x a trait-typed argument	No edge; recorded as dynamic dispatch
<code>with-all-assets-unsafe</code> ; <code>as-contract</code> around an external call; trait-typed call inside <code>as-contract?</code>	No edge; shown as a signal (Table 4)

300 honest wallets authorizes one to three entry contracts (router, vault, DAO or a token) with values drawn from a log-normal distribution, and pays fees for $1 + \text{Poisson}(8)$ transactions, each costing a log-normal multiple of the minimum fee with median three. Each router user makes one to five router calls, each binding the two tokens of one pool. A deployment costs 10 minimum fees. Ranks are counted among all contracts, including the attacker's, with ties resolved against the target. Table 8 gives the target's rank for several attack sizes.

Table 8: Rank of the attacker's token among all contracts in the synthetic example (1 is the top). k is the number of farm contracts (A1), Sybil wallets (A2) or router calls (A3).

Attack	k	Callers	Dep-PR	ER-uniform	PC-ER	PC-ER+bind
A1	0	10	14	15	15	15
A1	10	10	2	15	15	15
A1	100	10	1	2	15	15
A1	1,000	10	1	2	15	15
A2	0	10	14	15	15	15
A2	10	10	14	15	15	15
A2	100	3	14	2	15	15
A2	1,000	1	14	2	7	8
A3	0	10	14	15	15	15
A3	10	10	14	15	15	15
A3	100	10	14	15	15	15
A3	1,000	10	14	15	15	9